

Clase 2: Transformar y Manipular Datos

Análisis de Datos y Estadística Computacional

Prof. Gabriel Duarte Zúñiga

Universidad Adolfo Ibáñez | DIAPP-2026

13 de junio de 2026

¡Bienvenid@s a la Clase 2!

Resumen Clase 1

Función	Paquete	¿Para qué sirve?
<code>read_csv()</code> / <code>read_csv2()</code>	readr	Importar archivos <code>.csv</code> (separados por <code>,</code> o <code>;</code>)
<code>read_excel()</code>	readxl	Importar archivos Excel (<code>.xlsx</code> / <code>.xls</code>)
<code>read_dta()</code>	haven	Importar archivos de STATA (<code>.dta</code>)
<code>st_read()</code>	sf	Importar archivos espaciales (<code>.shp</code>)
<code>read_parquet()</code>	arrow	Importar archivos Parquet
<code>clean_names()</code>	janitor	Estandarizar nombres de columnas
<code>left_join()</code>	dplyr	Cruzar dos tablas por una llave

Resolución Ejercicio Grupal

El ejercicio pedía construir un dataset con la **población censada** y la **tasa de pobreza** para las 345 comunas del país:

```
1 library(tidyverse)
2 library(readxl)
3 library(janitor)
4
5 pobreza_2024 <- read_excel("data/SAE_ingresos_2024.xlsx",
6                             range = "A3:J348") |>
7   clean_names()
8
9 poblacion_2024 <- read_excel(
10   "data/D1_Poblacion-censada-por-sexo-y-edad-en-grupos-quinquenales.xlsx",
11   sheet = "2", skip = 3, n_max = 347) |>
12   clean_names()
13
14 datos <- left_join(pobreza_2024, poblacion_2024,
15                   join_by(codigo == codigo_comuna))
```

Nuestro dataset de trabajo

A lo largo de esta clase usaremos el dataset `datos` construido en la Clase 1 (345 comunas con datos de pobreza y población):

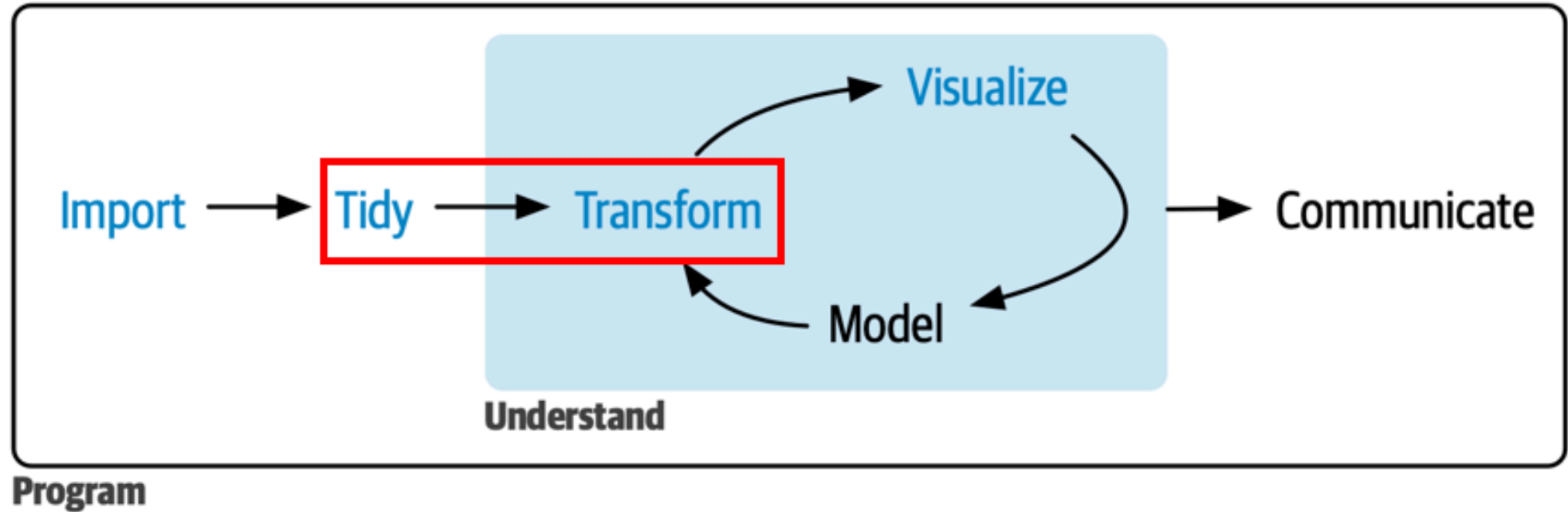
```
1 datos <- read_excel('data/clean/pobreza_pob_censo_2024.xlsx') |>
2   clean_names()
3 glimpse(datos)
```

Rows: 345

Columns: 19

\$ codigo	<dbl> 1101, ...
\$ region_x	<chr> "Tarap...
\$ nombre_comuna	<chr> "Iquiq...
\$ numero_de_personas_segun_proyecciones_de_poblacion	<dbl> 232455...
\$ numero_de_personas_en_situacion_de_pobreza_de_ingresos	<dbl> 37598...
\$ porcentaje_de_personas_en_situacion_de_pobreza_de_ingresos_2024	<dbl> 0.1617...
\$ limite_inferior	<dbl> 0.1478...
\$ limite_superior	<dbl> 0.1756...
\$ presencia_de_la_comuna_en_la_muestra_casen	<chr> "Sí", ...
\$ tipo_de_estimacion_sae	<chr> "Direc...
\$ codigo_region	<dbl> 1, 1, ...
\$ region_y	<chr> "Tarap...
\$ codigo_provincia	<dbl> 11, 11...
\$ provincia	<chr> "Iquiq...
\$ comuna	<chr> "Iquiq...
\$ poblacion_censada	<dbl> 199587

¿Qué veremos hoy?



Transformar y Manipular

Objetivo Clase 2

En esta segunda clase aprenderemos a **transformar y manipular** bases de datos utilizando los verbos del paquete `dplyr` y herramientas de `tidyr`.

Como objetivos específicos se busca:

- ▶ Reestructurar datos con `bind_rows()`, `pivot_longer()` y `pivot_wider()`.
- ▶ Ordenar filas con `arrange()`, filtrarlas con `filter()` y `filter_out()`, y extraer subconjuntos con la familia `slice()`.
- ▶ Seleccionar, reubicar, renombrar y crear variables con `select()`, `relocate()`, `rename()` y `mutate()`.
- ▶ Crear variables condicionales con `if_else()`, `case_when()` y las nuevas funciones `recode_values()` y `replace_when()`.
- ▶ Resumir estadísticas por grupos con `group_by()` + `summarise()` y `count()`.
- ▶ Manejar datos especiales: texto, fechas y factores.

Reestructurar Datos

Pegar filas: `bind_rows()`

- `bind_rows()` permite **apilar** dos o más tablas que tienen las mismas columnas (o al menos las mismas variables de interés):

```
1 norte <- tibble(region = c("Arica y Parinacota", "Tarapacá", "Antofagasta"),
2                   comunas = c(4, 7, 9))
3 sur   <- tibble(region = c("Araucanía", "Los Ríos", "Los Lagos"),
4                   comunas = c(32, 12, 30))
5
6 bind_rows(norte, sur)
```

A tibble: 6 × 2

	region <chr>	comunas <dbl>
1	Arica y Parinacota	4
2	Tarapacá	7
3	Antofagasta	9
4	Araucanía	32
5	Los Ríos	12
6	Los Lagos	30



Tip

A diferencia de `left_join()` (que pega **columnas**), `bind_rows()` pega **filas**: piénsenlo como apilar planillas una debajo de otra. Si las columnas no coinciden exactamente, `bind_rows()` rellena con `NA`.

Tidy data

Los datos están en formato *tidy* cuando:

- ▶ cada **variable** está en su propia **columna**,
- ▶ cada **observación** está en su propia **fila**, y
- ▶ cada **valor** está en su propia **celda**.

country	year	cases	population
Afghanistan	1999	17845	199947071
Afghanistan	2000	18666	200095360
Brazil	1999	31737	172006362
Brazil	2000	80488	174004898
China	1999	211258	1272015272
China	2000	216766	1280028583

variables

country	year	cases	population
Afghanistan	1999	17845	199947071
Afghanistan	2000	18666	200095360
Brazil	1999	31737	172006362
Brazil	2000	80488	174004898
China	1999	211258	1272015272
China	2000	216766	1280028583

observations

country	year	cases	population
Afghanistan	1999	17845	199947071
Afghanistan	2000	18666	200095360
Brazil	1999	31737	172006362
Brazil	2000	80488	174004898
China	1999	211258	1272015272
China	2000	216766	1280028583

values

Datos ordenados en formato tidy

Alargar el df: `pivot_longer()`

Cuando los nombres de columna son en realidad **valores** de una variable, necesitamos alargar la tabla. Esto lo podemos hacer mediante la función `pivot_longer()` para **ordenar verticalmente o sumarle filas** a nuestro dataset. Los argumentos principales requeridos son:

1. `cols = "vars"` Las columnas (variables) a pivotear.
2. `names_to = "x"` para indicar el **nombre** de la variable a donde irán las columnas pivote en 1.
3. `values_to = "y"` para indicar el **nombre** de la columna que almacenará los **valores** que contenían las antiguas columnas pivote.

Ejemplo: `pivot_longer()`

```
1 table4a
```

```
# A tibble: 3 × 3  
  country    `1999` `2000`  
  <chr>      <dbl> <dbl>  
1 Afghanistan    745    2666  
2 Brazil        37737   80488  
3 China         212258  213766
```

```
1 table4a |>  
2   pivot_longer(c(`1999`, `2000`),  
3                 names_to = "year",  
4                 values_to = "cases")
```

```
# A tibble: 6 × 3  
  country    year    cases  
  <chr>      <chr> <dbl>  
1 Afghanistan 1999     745  
2 Afghanistan 2000    2666  
3 Brazil      1999   37737  
4 Brazil      2000   80488  
5 China       1999  212258  
6 China       2000  213766
```

Ensanchar el df: `pivot_wider()`

Cuando una misma observación está repartida en **varias filas**, necesitamos ensanchar la tabla o **agregarle columnas**. Los argumentos principales requeridos son:

1. `names_from = "x"` que indica dónde se almacenan los **nombres de las nuevas columnas** que se generarán a partir del ensanchamiento.
2. `values_from = "y"` que indica desde dónde se obtendrán los **valores que tomarán** las observaciones de las nuevas columnas.

```
1 table2 |> head(4)
```

```
# A tibble: 4 × 4
  country    year type      count
  <chr>      <dbl> <chr>      <dbl>
1 Afghanistan 1999 cases         745
2 Afghanistan 1999 population 19987071
3 Afghanistan 2000 cases         2666
4 Afghanistan 2000 population 20595360
```

```
1 table2 |>
2   pivot_wider(names_from = type,
3               values_from = count)
```

```
# A tibble: 6 × 4
  country    year cases population
  <chr>      <dbl> <dbl>      <dbl>
1 Afghanistan 1999     745  19987071
2 Afghanistan 2000    2666  20595360
3 Brazil      1999   37737  172006362
4 Brazil      2000   80488  174504898
5 China       1999  212258  1272915272
6 China       2000  213766  1280428583
```

Tipos de variables con `glimpse()`

- Recordar que `glimpse()` muestra la clase de cada columna y algunas observaciones. Las abreviaciones más comunes son:

Abreviación	Clase	Ejemplo
<code><int></code>	Número entero (<i>integer</i>)	1, 2, 3
<code><dbl></code>	Número real (<i>double</i>)	3.14, 0.05
<code><chr></code>	Texto (<i>character</i>)	"Santiago"
<code><fct></code>	Factor (categórico)	Norte, Centro, Sur
<code><date></code>	Fecha	2026-06-16
<code><dtm></code>	Fecha y hora (<i>datetime</i>)	2026-06-16 09:00

Transformar Datos con dplyr

Consideraciones básicas

Las funciones de `dplyr` comparten estas características:

1. El **primer argumento** es un *dataframe*.
2. Los argumentos siguientes indican **sobre qué variable** operar (generalmente **sin comillas**).
3. El **resultado** siempre es un *dataframe*.
4. Se organizan en 3 grandes grupos:
 - ▶ **Filas:** `arrange()`, `filter()`, `filter_out()`, `slice()`
 - ▶ **Columnas:** `select()`, `relocate()`, `rename()`, `mutate()`
 - ▶ **Grupos:** `group_by()` + `summarise()`, `count()`

Filas

Ordenar: `arrange()`

- ▶ `arrange()` ordena las filas de un *dataframe* por los valores de una o más columnas.
- ▶ Por defecto ordena de forma **ascendente**. Para ordenar de manera **descendente** se usa `desc()` o un `-` antes de la variable numérica.

```
1 datos |>
2   arrange(desc(poblacion_censada)) |>
3   select(nombre_comuna, region_x, poblacion_censada) |>
4   head(5)
```

A tibble: 5 × 3

	nombre_comuna	region_x	poblacion_censada
	<chr>	<chr>	<dbl>
1	Puente Alto	Metropolitana	568086
2	Maipú	Metropolitana	503635
3	Santiago	Metropolitana	438856
4	Antofagasta	Antofagasta	401096
5	La Florida	Metropolitana	374836

Filtrar filas: `filter()`

- `filter()` conserva las filas que **cumplen todas** las condiciones indicadas. Las condiciones se separan por coma (equivale a utilizar `&` entre ellas).

```
1 datos |>
2   filter(region_x == "Metropolitana",
3         poblacion_censada > 200000) |>
4   select(nombre_comuna, poblacion_censada)
```

A tibble: 10 × 2

	nombre_comuna	poblacion_censada
	<chr>	<dbl>
1	Santiago	438856
2	La Florida	374836
3	Las Condes	296134
4	Maipú	503635
5	Ñuñoa	241467
6	Peñalolén	236478
7	Pudahuel	227820
8	Quilicura	205624
9	Puente Alto	568086
10	San Bernardo	306371

Filtrar filas: `filter()`

Operadores y funciones útiles para `filter()`:

Operador / Función	Descripción	Ejemplo
<code>==, !=</code>	Igual / Distinto	<code>region_x == "Biobío"</code>
<code>>, >=, <, <=</code>	Comparaciones	<code>poblacion_censada > 50000</code>
<code>%in%</code>	Pertenece a un conjunto	<code>region_x %in% c("Biobío", "Ñuble")</code>
<code>between(x, a, b)</code>	Rango cerrado	<code>between(poblacion_censada, 10000, 50000)</code>
<code>is.na(x)</code>	Es valor faltante	<code>filter(!is.na(poblacion_censada))</code>

Errores típicos al usar `filter()`

Error 1: usar `=` en vez de `==`

```
1 # ✗ Incorrecto
2 datos |>
3   filter(region_x = "Atacama")
```

```
Error in `filter()`:  
! We detected a named input.  
i This usually means that you've used `=`  
  instead of `==`.  
i Did you mean `region_x == "Atacama"`?
```

```
1 # ✓ Correcto
2 datos |>
3   filter(region_x == "Atacama")
```

Error 2: no repetir el nombre de la variable cuando se hacen condiciones adicionales (por ejemplo, con `|`)

```
1 # ✗ Incorrecto
2 datos |>
3   filter(region_x == "Atacama" | "Ñuble")
```

```
Error in `filter()`:  
i In argument: `region_x == "Atacama" |  
  "Ñuble"`.  
Caused by error in `region_x == "Atacama" |  
  "Ñuble"`:  
! solo son posibles operaciones para  
  variables de tipo numérico, compleja o  
  lógico
```

Errores típicos al usar `filter()`

```

1 #  Correcto
2 datos |>
3   filter(region_x == "Atacama" | region_x == "Ñuble") |>
4   head(3)

```

```

# A tibble: 3 × 19
  codigo region_x nombre_comuna  numero_de_personas_se...1 numero_de_personas_e...2
  <dbl> <chr>      <chr>                <dbl>                <dbl>
1   3101 Atacama Copiapó                176447                29286.
2   3102 Atacama Caldera                  19985                 4417.
3   3103 Atacama Tierra Amarilla          14434                 3405.
# i abbreviated names: 1numero_de_personas_según_proyecciones_de_población,
#   2numero_de_personas_en_situación_de_pobreza_de_ingresos
# i 14 more variables:
#   porcentaje_de_personas_en_situación_de_pobreza_de_ingresos_2024 <dbl>,
#   limite_inferior <dbl>, limite_superior <dbl>,
#   presencia_de_la_comuna_en_la_muestra_casen <chr>,
#   tipo_de_estimacion_sae <chr>, codigo_region <dbl>, region_y <chr>, ...

```

```

1 # O lo mismo pero más eficiente
2 datos |>
3   filter(region_x %in% c("Atacama", "Ñuble"))

```

Excluir filas: `filter_out()`

- **Novedad de dplyr 1.2.0:** `filter_out()` es el complemento de `filter()`. Mientras `filter()` **conserva** las filas que cumplen la condición, `filter_out()` las **descarta**.
- La diferencia clave está en cómo tratan los **NA**: `filter()` los descarta, pero `filter_out()` los **conserva**:

```
1 autos <- tibble(clase = c("suv", NA, "sedan"),
2                 precio = c(30, 25, 20))
3
4 # filter() descarta NA también
5 autos |> filter(clase != "suv")
```

```
# A tibble: 1 × 2
  clase precio
<chr>   <dbl>
1 sedan     20
```

```
1 # filter_out() conserva NA (solo descarta "suv")
2 autos |> filter_out(clase == "suv")
```

```
# A tibble: 2 × 2
  clase precio
<chr>   <dbl>
```

filter() vs filter_out()

filter() → para **conservar** filas

```
1 datos |>  
2   filter(poblacion_censada > 100000)
```

- ▶ Trata **NA** como **FALSE** → los **descarta**.
- ▶ Ideal cuando la intención es “**quedarse con las filas que cumplan X**”.

filter_out() → para **descartar** filas

```
1 datos |>  
2   filter_out(poblacion_censada < 5000)
```

- ▶ Trata **NA** como **FALSE** → los **conserva**.
- ▶ Ideal cuando la intención es “**eliminar las filas que cumplan X**”.



Tip

Regla simple: si estás usando filter() con **!=** o **!** y además necesitas agregar **| is.na(...)**, probablemente **filter_out()** sea más claro y directo.



Pregunta N° 1

Tenemos un dataset de comunas donde la variable `tasa_pobreza` tiene algunos valores `NA`. Queremos **eliminar** las comunas con tasa de pobreza superior al 30%, pero **sin perder** las comunas con `NA`.

¿Cuál es la mejor opción?

```
filter(tasa_pobreza <= 0.30)
```

```
filter(tasa_pobreza <= 0.30 | is.na(tasa_pobreza))
```

```
filter_out(tasa_pobreza > 0.30)
```

```
filter_out(tasa_pobreza <= 0.30)
```

[Revisar](#)[Reiniciar](#)

Extraer filas: familia `slice()`

`slice()` selecciona filas por **posición**. Sus variantes más útiles son:

Función	Descripción
<code>slice_head(n = 5)</code>	Primeras 5 filas
<code>slice_max(x, n = 5)</code>	5 filas con mayor valor de <code>x</code>
<code>slice_min(x, n = 5)</code>	5 filas con menor valor de <code>x</code>
<code>slice_sample(n = 5)</code>	5 filas al azar

```
1 datos |>
2   slice_max(poblacion_censada, n = 3) |>
3   select(nombre_comuna, region_x, poblacion_censada)
```

```
# A tibble: 3 × 3
  nombre_comuna region_x      poblacion_censada
  <chr>         <chr>         <dbl>
1 Puente Alto  Metropolitana  568086
2 Maipú        Metropolitana  503635
3 Santiago     Metropolitana  438856
```

slice() combinado con group_by()

Podemos obtener las comunas más pobladas **por región**:

```
1 datos |>
2   group_by(region_x) |>
3   slice_max(poblacion_censada, n = 1) |>
4   ungroup() |>
5   select(region_x, nombre_comuna, poblacion_censada) |>
6   head(5)
```

A tibble: 5 × 3

	region_x	nombre_comuna	poblacion_censada
	<chr>	<chr>	<dbl>
1	Antofagasta	Antofagasta	401096
2	Arica y Parinacota	Arica	241653
3	Atacama	Copiapó	168831
4	Aysén	Coyhaique	57823
5	Biobío	Concepción	230375



Filtrar y ordenar

Usando el dataset `gapminder`, primero resuma la estructura del dataset con `glimpse()` y luego encuentre los **3 países del continente (continent) Americas** con mayor esperanza de vida (`lifeExp`) en el **año 2007**.

R Code [↺ Start Over](#)

[▶ Run Code](#)

```
1 library(gapminder)
2 library(dplyr)
3
4 _____(gapminder)
```

R Code [↺ Start Over](#)

[▶ Run Code](#)

```
1 library(gapminder)
2 library(dplyr)
3
4 gapminder |>
5   filter(_____ & _____) |>
6   slice_max(_____, n = _____) |>
7   select(country, lifeExp, gdpPercap)
```

Columnas

Seleccionar: `select()`

- `select()` conserva solo las columnas indicadas. Acepta funciones auxiliares (*helpers*) para seleccionar por patrón:

Helper	Descripción	Ejemplo
<code>starts_with("x")</code>	Comienza con "x"	<code>starts_with("codigo")</code>
<code>ends_with("x")</code>	Termina con "x"	<code>ends_with("comuna")</code>
<code>contains("x")</code>	Contiene "x"	<code>contains("pobreza")</code>
<code>where(is.numeric)</code>	Por tipo de variable	<code>where(is.character)</code>

```
1 datos |>
2   select(nombre_comuna, starts_with("poblacion")) |>
3   head(3)
```

```
# A tibble: 3 × 2
  nombre_comuna poblacion_censada
  <chr>          <dbl>
1 Iquique       199587
2 Alto Hospicio 142086
3 Pozo Almonte  16878
```

Reubicar y renombrar

Crear variables: `mutate()`

- `mutate()` crea nuevas columnas o modifica existentes:

```
1 datos |>
2   mutate(
3     pob_miles = round(poblacion_censada / 1000, 1),
4     prop_hombres = round(hombres / poblacion_censada, 3)
5   ) |>
6   select(nombre_comuna, pob_miles, prop_hombres) |>
7   head(4)
```

```
# A tibble: 4 × 3
  nombre_comuna pob_miles prop_hombres
  <chr>         <dbl>     <dbl>
1 Iquique       200.       0.493
2 Alto Hospicio 142.       0.497
3 Pozo Almonte  16.9       0.514
4 Camiña        1.3       0.485
```


Aplicar funciones a múltiples columnas: `across()`

- ▶ `across()` permite aplicar una misma función a **varias columnas** a la vez, usando los mismos *helpers* de `select()`:

```
1 datos |>
2   summarise(
3     across(c(poblacion_censada, hombres, mujeres),
4             \(x) round(mean(x, na.rm = TRUE)))
5   )
```

```
# A tibble: 1 × 3
  poblacion_censada hombres mujeres
      <dbl>      <dbl>   <dbl>
1         53566      25991   27575
```



Tip

`across()` se usa dentro de `mutate()` o `summarise()`. Es especialmente útil cuando necesitas aplicar la misma transformación a muchas columnas: `across(where(is.numeric), \(x) round(x, 2))`.

Condiciones binarias: `if_else()`

- `if_else()` evalúa una condición y asigna un valor para `TRUE` y otro para `FALSE`:

```
1 datos |>
2   mutate(
3     tamano = if_else(poblacion_censada > 100000,
4                     "Grande", "Pequeña")
5   ) |>
6   select(nombre_comuna, poblacion_censada, tamano) |>
7   head(4)
```

```
# A tibble: 4 × 3
  nombre_comuna poblacion_censada tamano
  <chr>          <dbl> <chr>
1 Iquique       199587 Grande
2 Alto Hospicio 142086 Grande
3 Pozo Almonte  16878 Pequeña
4 Camiña        1335 Pequeña
```



Tip

`if_else()` es la opción correcta cuando hay exactamente **dos categorías**. Para más categorías, usar `case_when()`.

Múltiples condiciones: `case_when()`

- `case_when()` evalúa condiciones **en orden** y asigna el valor de la primera que se cumple:

```
1 datos |>
2   mutate(
3     tamaño = case_when(
4       poblacion_censada > 200000 ~ "Metrópoli",
5       poblacion_censada > 50000  ~ "Ciudad grande",
6       poblacion_censada > 10000  ~ "Ciudad mediana",
7       .default = "Pueblo"
8     )
9   ) |>
10  count(tamaño)
```

A tibble: 4 × 2

	tamaño	n
	<chr>	<int>
1	Ciudad grande	68
2	Ciudad mediana	165
3	Metrópoli	22
4	Pueblo	90

Recodificar: `recode_values()`

- **Novedad de dplyr 1.2.0:** cuando la recodificación es un mapeo directo de valores (como una tabla de búsqueda), `recode_values()` es más limpio que `case_when()`:

```
1 # Con case_when (verboso)
2 datos |>
3   mutate(zona = case_when(
4     codigo_region == 15 ~ "Norte",
5     codigo_region == 1  ~ "Norte",
6     codigo_region == 13 ~ "Centro",
7     .default = "Otro"
8   ))
9
10 # Con recode_values (más directo, usando tabla de búsqueda)
11 lookup <- tribble(
12   ~from, ~to,
13   15,    "Norte",
14   1,     "Norte",
15   13,    "Centro"
16 )
17
18 datos |>
19   mutate(zona = recode_values(codigo_region,
20                               from = lookup$from,
21                               to = lookup$to,
```

Reemplazar parcialmente: `replace_when()`

- `replace_when()` es para **modificar solo algunos valores** de una columna existente, manteniendo el resto intacto:

```
1 # Ejemplo: truncar la población a un máximo de 500.000
2 datos |>
3   mutate(
4     poblacion_censada = replace_when(
5       poblacion_censada,
6       poblacion_censada > 500000 ~ 500000
7     )
8   )
```



Tip

La diferencia con `case_when()` es que `replace_when()` **no necesita un `.default`**: los valores que no cumplen la condición simplemente se mantienen como están. Esto es más seguro cuando solo quieres tocar algunas filas.



Pregunta N° 2

Queremos crear una nueva variable `zona` que clasifique las comunas según la región: Norte (regiones 1 a 4 y 15), Centro (5 y 13) y Sur (el resto). ¿Cuál es el **mejor** enfoque?

¿Qué función usarías?

```
if_else()
```

```
case_when()
```

```
recode_values()
```

```
replace_when()
```

Revisar

Reiniciar



Crear y transformar

Usando `gapminder`, filtre el año **2007** y cree una variable `nivel_desarrollo` que clasifique los países según su PIB per cápita: “Alto” si supera los 20.000, “Medio” entre 5.000 y 20.000, y “Bajo” en caso contrario. Luego cuente cuántos países hay en cada nivel.

R Code

[↺ Start Over](#)[▶ Run Code](#)

```
1 library(gapminder)
2 library(dplyr)
3
4 gapminder |>
5   filter(year == 2007) |>
6   mutate(
7     nivel_desarrollo = case_when(
8       gdpPercap > ____ ~ "____",
9       gdpPercap > ____ ~ "____",
10      .default = "____"
11    )
12  ) |>
13  count(____)
```

Agrupamiento

group_by() + summarise()

- ▶ `group_by()` **prepara** el *dataframe* para realizar operaciones por grupo. No cambia los datos por sí solo.
- ▶ `summarise()` **calcula** estadísticas de resumen para cada grupo:

```
1 datos |>
2   group_by(region_x) |>
3   summarise(
4     n_comunas = n(),
5     pob_total = sum(poblacion_censada),
6     pob_media = round(mean(poblacion_censada))
7   ) |>
8   ungroup() |>
9   slice_max(pob_total, n = 5)
```

A tibble: 5 × 4

	region_x <chr>	n_comunas <int>	pob_total <dbl>	pob_media <dbl>
1	Metropolitana	52	7400741	142322
2	Valparaíso	38	1896053	49896
3	Biobío	33	1613059	48881
4	Maule	30	1123008	37434
5	La Araucanía	32	1010423	31576

group_by() + summarise(): precauciones

Algunas consideraciones importantes:

- ▶ `group_by()` prepara el *dataframe* pero **no** lo modifica visualmente.
- ▶ Notar que es posible agrupar por **más de una variable**:
- ▶ `summarise()` reduce el *dataframe* a **una fila por grupo**.
- ▶ Si se desea seguir manipulando el resultado **sin agrupación**, siempre encadenar `ungroup()` al final. De lo contrario, las operaciones posteriores seguirán una lógica agrupada.
- ▶ `.groups = "drop"` dentro de `summarise()` hace explícito que el resultado final ya no tiene grupos: *“Una vez calculado el resumen con summarise(), elimina todos los agrupamientos para que el resultado se devuelva como una tabla sin grupos”*:

```
1 datos |>
2   group_by(region_x, presencia_de_la_comuna_en_la_muestra_casen) |>
3   summarise(n = n(), .groups = "drop") |>
4   head(4)
```

A tibble: 4 × 3

	region_x <chr>	presencia_de_la_comuna_en_la_muestra_casen <chr>	n <int>
1	Antofagasta	No	1
2	Antofagasta	Sí	8

Contar: `count()`

- `count()` es un atajo para `group_by() |> summarise(n = n())`:

```
1 datos |>  
2   count(region_x, name = "n_comunas") |>  
3   slice_max(n_comunas, n = 5)
```

```
# A tibble: 5 × 2  
  region_x      n_comunas  
  <chr>         <int>  
1 Metropolitana      52  
2 Valparaíso        38  
3 Biobío            33  
4 O'Higgins         33  
5 La Araucanía      32
```

Contar: `count()`

- ▶ Con el argumento `wt` podemos **sumar** una variable en vez de contar filas, mientras que con el argumento `name` se puede cambiar el nombre de la variable de recuento (o suma):

```
1 datos |>
2   count(region_x, wt = poblacion_censada,
3         name = "pob_total") |>
4   slice_max(pob_total, n = 3)
```

```
# A tibble: 3 × 2
  region_x      pob_total
  <chr>         <dbl>
1 Metropolitana 7400741
2 Valparaíso    1896053
3 Biobío        1613059
```

Ejercicio Grupal (30-40 min)

Trabajarán en **salas pequeñas** usando el dataset `pobreza_pob_censo_2024.xlsx` y otros datos homicidios. Los detalles están en la **Guía de Ejercicio Grupal N° 2** disponible en [Webcursos](#).



Recuerden

- ▶ Designen a una persona que comparta pantalla.
- ▶ Intenten resolver primero y luego revisen la pauta.
- ▶ El profesor y ayudante pasarán por las salas.

Datos Especiales

Texto

Extraer partes de texto: `str_sub()`

- ▶ `str_sub()` del paquete `stringr` permite extraer una **porción** de una cadena de texto según sus posiciones:

```
1 datos |>
2   mutate(
3     cod_region = str_sub(as.character(codigo), 1, 2)
4   ) |>
5   select(codigo, nombre_comuna, cod_region) |>
6   head(4)
```

```
# A tibble: 4 × 3
  codigo nombre_comuna cod_region
  <dbl> <chr>          <chr>
1  1101 Iquique      11
2  1107 Alto Hospicio 11
3  1401 Pozo Almonte  14
4  1402 Camiña       14
```

- ▶ Argumentos: `str_sub(string, start, end)`. Las posiciones son **inclusivas** (1-indexed).

Aplicación: extraer períodos desde texto

Un caso muy frecuente en datos públicos: variables que codifican trimestre y año juntos. Notar que también se pueden usar posiciones **negativas** anteponiendo un menos al segundo argumento para contar desde el **final** de la cadena. Por ejemplo `str_sub(periodo, -2)` extrae los últimos 2 caracteres:

```
1 trimestres <- tibble(
2   periodo = c("tr1_2023", "tr2_2023", "tr3_2023", "tr4_2023"),
3   ventas  = c(1500, 2000, 3500, 1750)
4 )
5
6 trimestres |>
7   mutate(
8     trimestre = str_sub(periodo, 3, 3),
9     anio      = str_sub(periodo, -4)
10  )
```

```
# A tibble: 4 × 4
  periodo  ventas trimestre anio
  <chr>    <dbl>   <chr>   <chr>
1 tr1_2023  1500     1       2023
2 tr2_2023  2000     2       2023
3 tr3_2023  3500     3       2023
4 tr4_2023  1750     4       2023
```

Fechas

Fechas con **lubridate**

- El paquete **lubridate** (parte del **tidyverse**) facilita el manejo de fechas. Las funciones **ymd()**, **dmy()**, **mdy()** convierten texto a fecha según el **orden** de los componentes:

```
1 ymd("2026-06-16")
```

```
[1] "2026-06-16"
```

```
1 dmy("16-junio-2026")
```

```
[1] "2026-06-16"
```

```
1 mdy("June 16, 2026")
```

```
[1] "2026-06-16"
```



Tip

La norma ISO 8601 ordena las fechas de mayor a menor: **AAAA-MM-DD**. Usar **ymd()** cuando los datos siguen este formato, **dmy()** cuando están en formato chileno (día/mes/año).

Extraer componentes de una fecha

Una vez que la variable es de clase `<date>`, podemos extraer sus partes:

```
1 fechas <- tibble(  
2   fecha = ymd(c("2026-01-15", "2026-06-22", "2026-12-01"))  
3 )  
4  
5 fechas |>  
6   mutate(  
7     anio = year(fecha),  
8     mes  = month(fecha, label = TRUE),  
9     dia  = day(fecha),  
10    dia_semana = wday(fecha, label = TRUE)  
11  )
```

```
# A tibble: 3 × 5  
  fecha      anio mes    dia dia_semana  
  <date>    <dbl> <ord> <int> <ord>  
1 2026-01-15  2026 ene     15 jue  
2 2026-06-22  2026 jun     22 lun  
3 2026-12-01  2026 dic      1 mar
```

Convertir texto a fecha: `as_date()`

- ▶ Cuando las fechas están almacenadas como texto (clase `<chr>`), debemos convertirlas con `as_date()` o las funciones `ymd()`/`dmy()`:

```
1 df_texto <- tibble(  
2   fecha_texto = c("2026-01-15", "2026-06-22", "2026-12-01")  
3 )  
4 glimpse(df_texto)    # <chr>
```

Rows: 3

Columns: 1

\$ fecha_texto <chr> "2026-01-15", "2026-06-22", "2026-12-01"

```
1 df_fecha <- df_texto |>  
2   mutate(fecha = as_date(fecha_texto))  
3 glimpse(df_fecha)    # ahora <date>
```

Rows: 3

Columns: 2

\$ fecha_texto <chr> "2026-01-15", "2026-06-22", "2026-12-01"

\$ fecha <date> 2026-01-15, 2026-06-22, 2026-12-01

Calcular intervalos de tiempo

- Para calcular la diferencia entre dos fechas:

```
1 inicio <- ymd("2026-06-09")    # inicio del diplomado
2 termino <- ymd("2026-07-28")   # última clase
3 termino - inicio                # diferencia en días
```

Time difference of 49 days

```
1 # Intervalos más expresivos con interval()
2 intervalo <- interval(inicio, termino)
3 as.duration(intervalo)
```

```
[1] "4233600s (~7 weeks)"
```



Pregunta N° 3

Tenemos una variable `fecha_texto` con valores como `"15/06/2026"` (en formato día/mes/año, típico en Chile). ¿Cómo la convertimos correctamente a fecha?

¿Cuál es la función correcta?

```
ymd("15/06/2026")
```

```
dmy("15/06/2026")
```

```
mdy("15/06/2026")
```

```
as_date("15/06/2026")
```

Revisar

Reiniciar

Factores

Variables categóricas: `factor()`

- Un **factor** es una estructura de datos para representar variables categóricas con un número finito de **niveles**:

```
1 tamanos <- c("Grande", "Mediana", "Pequeña", "Grande", "Mediana")
2 factor(tamanos)
```

```
[1] Grande Mediana Pequeña Grande Mediana
Levels: Grande Mediana Pequeña
```

- Por defecto los niveles se ordenan **alfabéticamente**. Podemos especificar el orden con `levels`:

```
1 factor(tamanos, levels = c("Pequeña", "Mediana", "Grande"))
```

```
[1] Grande Mediana Pequeña Grande Mediana
Levels: Pequeña Mediana Grande
```

Reordenar niveles: `fct_relevel()`

- `fct_relevel()` del paquete `forcats` permite **reordenar** los niveles de un factor ya creado:

```
1 f <- factor(tamamos)
2 levels(f) # alfabético
```

```
[1] "Grande" "Mediana" "Pequeña"
```

```
1 f2 <- fct_relevel(f, "Pequeña", "Mediana", "Grande")
2 levels(f2) # orden personalizado
```

```
[1] "Pequeña" "Mediana" "Grande"
```



Tip

`fct_relevel()` es especialmente útil cuando queremos controlar el **orden de las categorías en los gráficos** de `ggplot2` (Clase 3).



Factores

Usando `gapminder` (año 2007), clasifique los países según su esperanza de vida en una variable numérica en función de un umbral de 70 años de edad (1 = alta (mayor igual a 70), 0 = baja (menor que 70)) y luego haga un recuento usando un factor con las etiquetas designadas (alta y baja).

R Code

[↺ Start Over](#)[▶ Run Code](#)

```
1 library(gapminder)
2 library(dplyr)
3 library(forcats)
4
5
6 gapminder |>
7   filter(year == 2007) |>
8   mutate(
9     nivel_vida = _____(lifeExp >= 70, 1, 0),
10    nivel_vida = factor(_____, levels = c(0, 1), labels = c("_____", "_____"))
11  ) |>
12  count(nivel_vida)
```

Recapitulación

¿Qué aprendimos hoy?

Grupo	Funciones
Reestructurar	<code>bind_rows()</code> , <code>pivot_longer()</code> , <code>pivot_wider()</code>
Filas	<code>arrange()</code> , <code>filter()</code> , <code>filter_out()</code> , <code>slice_max/min/sample()</code>
Columnas	<code>select()</code> , <code>relocate()</code> , <code>rename()</code> , <code>mutate()</code> , <code>across()</code>
Condicionales	<code>if_else()</code> , <code>case_when()</code> , <code>recode_values()</code> , <code>replace_when()</code>
Grupos	<code>group_by()</code> + <code>summarise()</code> , <code>count()</code>
Texto	<code>str_sub()</code>
Fechas	<code>ymd()</code> , <code>dmy()</code> , <code>as_date()</code> , <code>year()</code> , <code>month()</code> , <code>interval()</code>
Factores	<code>factor()</code> , <code>fct_relevel()</code>

La próxima clase aprenderemos a **visualizar** estos datos con `ggplot2` para comunicar patrones y resultados.